Page :
Date :

EXPERIMENT-07.

Ques : cheapest flight from source to destination with atmost k-stops.

Algorithm :-

1. Build the graph.
   ↳ converts the flights list into an adjacency list.
2. Initialize BFS queue :
   ↳ Each element : {current city, total cost so far, stops used}
   ↳ starts with {source, 0, 0}
3. Track minimum cost :
   ↳ create minCost[n] array to store the cheapest cost to reach each city.

4. BFS Traversal :
   while queue is not empty :
   ① pop the front element {city, cost, stops}.
   ② If stops > k, skip it
   ③ for each neighbour,
   if city + price < minCost [nextcity]
   then update and push into the queue.

5. Return Result
   If minCost [d] is still INT_MAX → ret. -1
   Else return minCost[d] (cheapest Cost.).

Code

T.C ⇒ O(E * K)
S.P ⇒ O(E+V).

```
#include <iostream>
#include <vector>
# include <queue>
using namespace std;

class Solution{
    int findCheapestPrice(int n, vector<vector<int>>& flights, int
        source, int d, int k){
        vector<vector<pair><int, int>>>graph(n);
        for (auto &flight : flights){
            graph[flight[0]].push_back{flight[1], flight[2]});
        }
        queue<vector<int>>q;
        q.push ~~[src]~~ ({source, 0, 0});
        vector<int> minCost (n, INT_MAX);
        minCost[source] = 0;
        while (!q.empty()){
            auto current = q.front();
            q.pop();
            int city = current[0], cost = current[1], stops = current[2];
            if (stops > k) continue;
            for (auto & neighbour : graph[city]){
                int nextCity = neighbour.first;
                int price = neighbour.second;
                if (cost + price < minCost[nextCity]){
                    minCost[nextCity] = cost+price;
                    q.push({nextCity, cost+price, stops+1});
                }
            }
        }
        return minCost[d] == INT_MAX ? -1 : minCost[d];
    }
```